Name:Khushi Singh
Class:D20C Roll no.34

Experiment No. 2

Aim: Create a Blockchain using Python

Theory:
Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using create_block() with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – is_chain_valid() checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes /mine_block, /get_chain, and /is_valid allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code:
1. Importing Required Libraries

- **datetime** → to timestamp each block
- **hashlib** → to generate SHA-256 hashes
- **json** → to encode blocks as JSON for hashing
- **Flask** → to run the blockchain API server

2. Blockchain Class – Core Logic
This contains everything needed to:
- create blocks
- store the chain
- implement Proof of Work
- validate chain integrity

3. Flask Web API
Provides endpoints:

- /mine_block → mine a new block

- /get_chain → fetch full blockchain
- /is_valid → verify blockchain

Code:

```
import datetime
import hashlib
import json
from flask import Flask, jsonify


class Blockchain:

    def __init__(self):
        self.chain = []
        self.transactions = []

        # Create Genesis Block
        self.create_block(proof=1,
previous_hash='0')

    def create_block(self, proof,
previous_hash, transactions=None):

        if transactions is None:
            transactions = []

        block = {
            'index': len(self.chain)
+ 1,
            'timestamp':
str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash':
previous_hash,
            'transactions':
transactions
        }

        self.chain.append(block)

        return block

    def get_previous_block(self):
        return self.chain[-1]

    def create_Transactions(self):

        # Mine only when transaction
list is not empty
        if self.transactions:

            # Get previous block
            previous_block =
self.get_previous_block()

            # Get previous proof
            previous_proof =
previous_block['proof']

            # Find golden nonce
            proof =
self.proof_of_work(previous_proof)

            # Get hash of previous
block
            previous_hash =
self.hash(previous_block)

            # Remove transactions
from pending list BEFORE mining
            transactions_to_mine =
self.transactions.copy()
            self.transactions = []

            # Create new block
containing transactions
            block =
self.create_block(
                proof,
                previous_hash,
                transactions_to_mine
            )

            return block

        # No transactions -> do not
mine
        return None

    def proof_of_work(self,
previous_proof):

        new_proof = 1
        check_proof = False

        while check_proof is False:
```

```
            hash_operation =
hashlib.sha256(
                str(new_proof**2 -
previous_proof**2).encode()
            ).hexdigest()

            # Cryptographic puzzle:
            # Hash must start with
000
            if hash_operation[:3] ==
'000':
                check_proof = True
            else:
                new_proof += 1

        return new_proof

    def hash(self, block):

        encoded_block = json.dumps(
            block,
            sort_keys=True
        ).encode()

        return
hashlib.sha256(encoded_block).hexdig
est()

    def is_chain_valid(self, chain):

        previous_block = chain[0]
        block_index = 1

        while block_index <
len(chain):

            block =
chain[block_index]

            # Check previous hash
            if
block['previous_hash'] !=
self.hash(previous_block):
                return False

            # Check Proof-of-Work
            previous_proof =
previous_block['proof']
            proof = block['proof']

            hash_operation =
hashlib.sha256(
                str(proof**2 -
previous_proof**2).encode()
            ).hexdigest()

            if hash_operation[:3] !=
'000':
                return False

            previous_block = block
            block_index += 1

        return True

# Flask Application

app = Flask(__name__)

blockchain = Blockchain()

# Mine Block

@app.route('/mine_block',
methods=['GET'])
def mine_block():

    previous_block =
blockchain.get_previous_block()

    previous_proof =
previous_block['proof']

    proof =
blockchain.proof_of_work(previous_pr
oof)

    previous_hash =
blockchain.hash(previous_block)

    block = blockchain.create_block(
        proof,
        previous_hash,
        blockchain.transactions
    )

    # Remove transactions after
adding them to block
    blockchain.transactions = []

    response = {
        'message': 'Congratulations,
you just mined a block!',
        'index': block['index'],
```

```
        'timestamp':
block['timestamp'],
        'proof': block['proof'],
        'previous_hash':
block['previous_hash'],
        'transactions':
block['transactions']
    }

    return jsonify(response), 200

# Get Blockchain

@app.route('/get_chain',
methods=['GET'])
def get_chain():

    response = {
        'chain': blockchain.chain,
        'length':
len(blockchain.chain)
    }

    return jsonify(response), 200

# Check Blockchain Validity

@app.route('/is_valid',
methods=['GET'])
def is_valid():

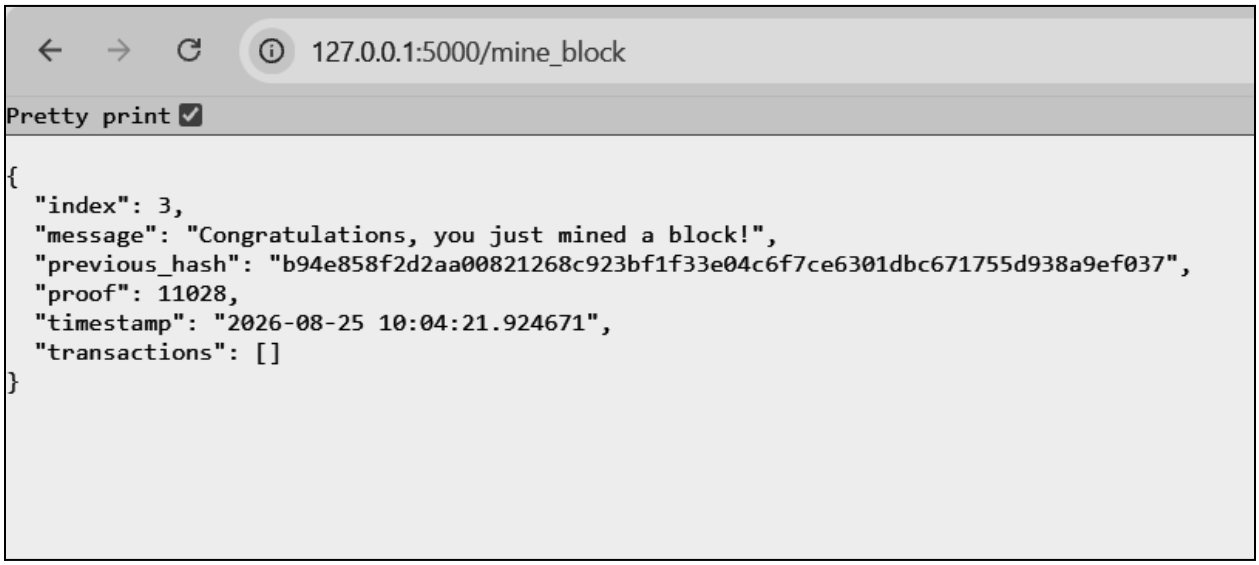
    valid =
blockchain.is_chain_valid(
        blockchain.chain
    )

    if valid:
        response = {
            'message': 'All good.
The Blockchain is valid.'
        }
    else:
        response = {
            'message': 'The
Blockchain is not valid.'
        }

    return jsonify(response), 200

# Run Flask Server

if __name__ == '__main__':
    app.run(host='0.0.0.0',
port=5000
```



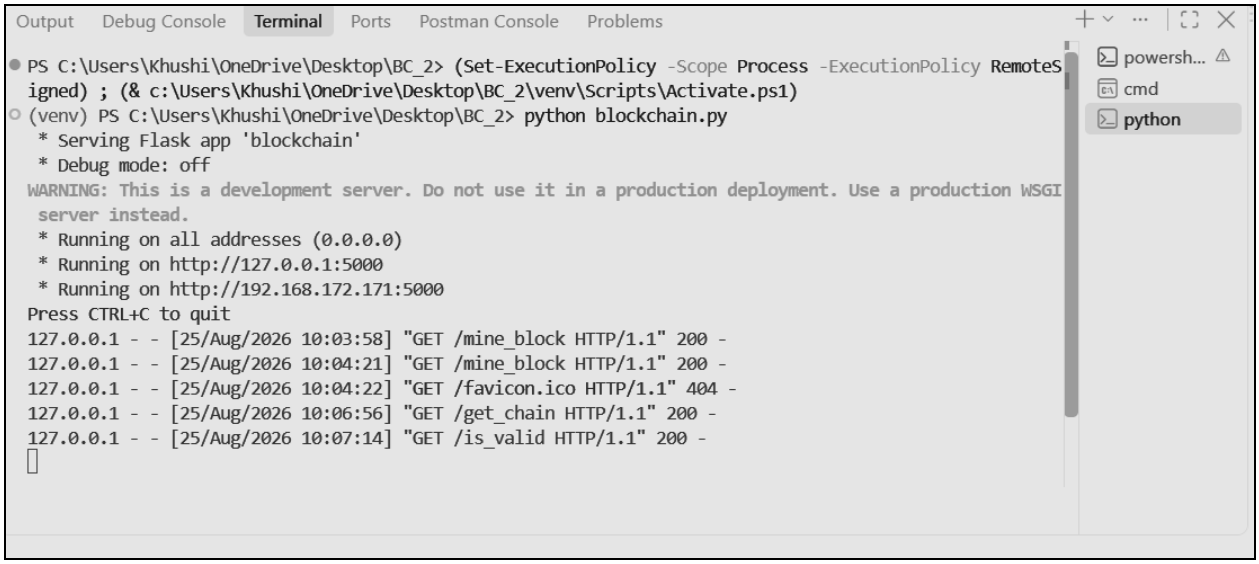



```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-08-25 10:03:26.289411",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "e5aa075557f3983951f52f3462fdc55884850bbceb114f56dd39958d142119d7",
      "proof": 533,
      "timestamp": "2026-08-25 10:03:58.068466",
      "transactions": []
    },
    {
      "index": 3,
      "previous_hash": "b94e858f2d2aa00821268c923bf1f33e04c6f7ce6301dbc671755d938a9ef037",
      "proof": 11028,
      "timestamp": "2026-08-25 10:04:21.924671",
      "transactions": []
    }
  ],
  "length": 3
}
```





```
{
  "message": "All good. The Blockchain is valid."
}
```



Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.